D-Flow: Differentiating through Flows for Controlled Generation

Topic 2 Group 1
Khaled Jebari
Mohamed Amine Oueslati
Sedki Gharsallah

IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

Summary

01

Considered Issue

02

Proposed Model

03

Implementation

04

Experiments

01

Considered issue

Considered issue

Flow based models lack the possibility of conditioning the generation

→ Particularly problematic when considering certain types of problems or when control of generation is necessary.

Conditional Generation

Inverse problems

The D-Flow article attempts to answer this question by providing a general method based on adjusting the initial sample selection to obtain the desired final sample.

02

Proposed model

Overview

We consider a pretrained generative flow (or diffusion-ODE) model that defines a velocity field

$$\dot{x}(t) = u_\theta(t, x(t)), \quad t \in [0, 1], \quad x(0) = x_0,$$

where the model parameters θ are **frozen**.

The goal of D-Flow is to solve the optimization problem

$$\min_{x_0} L(x(1)), + \quad \mathrm{R} \quad (\mathrm{R} = 0 \text{ in our case})$$

where $L(x)$ is a task-specific cost function.

The optimization variable is therefore the **initial noise** x_0.

State space formulation

$$x_0 \sim \mathcal{N}(0, I)$$

$$x_1 = \phi(x_0)$$

$$y = \mathcal{H}(x_1) + \varepsilon \qquad \varepsilon \sim \mathcal{N}(0, R)$$

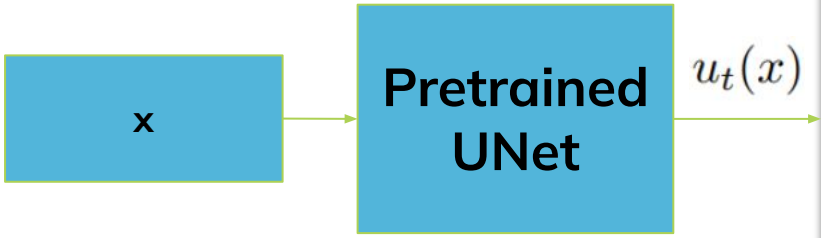
$$\mathcal{L}(x_0) = \frac{1}{2} \|\mathcal{H}(\phi(x_0)) - y\|^2_{R^{-1}} + \frac{\lambda}{2} \|x_0\|^2$$

03

Implementation

Implementation

$$\mathcal{L}(x) = \|H(x) - y\|^2$$



Algorithm 1 D-Flow.

Require: cost $\mathcal{L}$, pre-trained flow model $u_t(x)$
Initialize $x_0^{(0)} = x_0$
for $i = 1, \ldots, N$ **do**
  $x^{(i)}(1) \leftarrow \texttt{solve}(x_0^{(i)}, u_t)$
  $x_0^{(i+1)} \leftarrow \texttt{optimize_step}(x_0^{(i)}, \nabla_{x_0}\mathcal{L}(x^{(i)}(1)))$
end for
Return: $x^N(1)$

04

Experiments

Experiments

$$z_0^* = \arg\min_{z_0} \left\| \mathcal{A}\big(\mathrm{Flow}(z_0)\big) - y \right\|^2$$

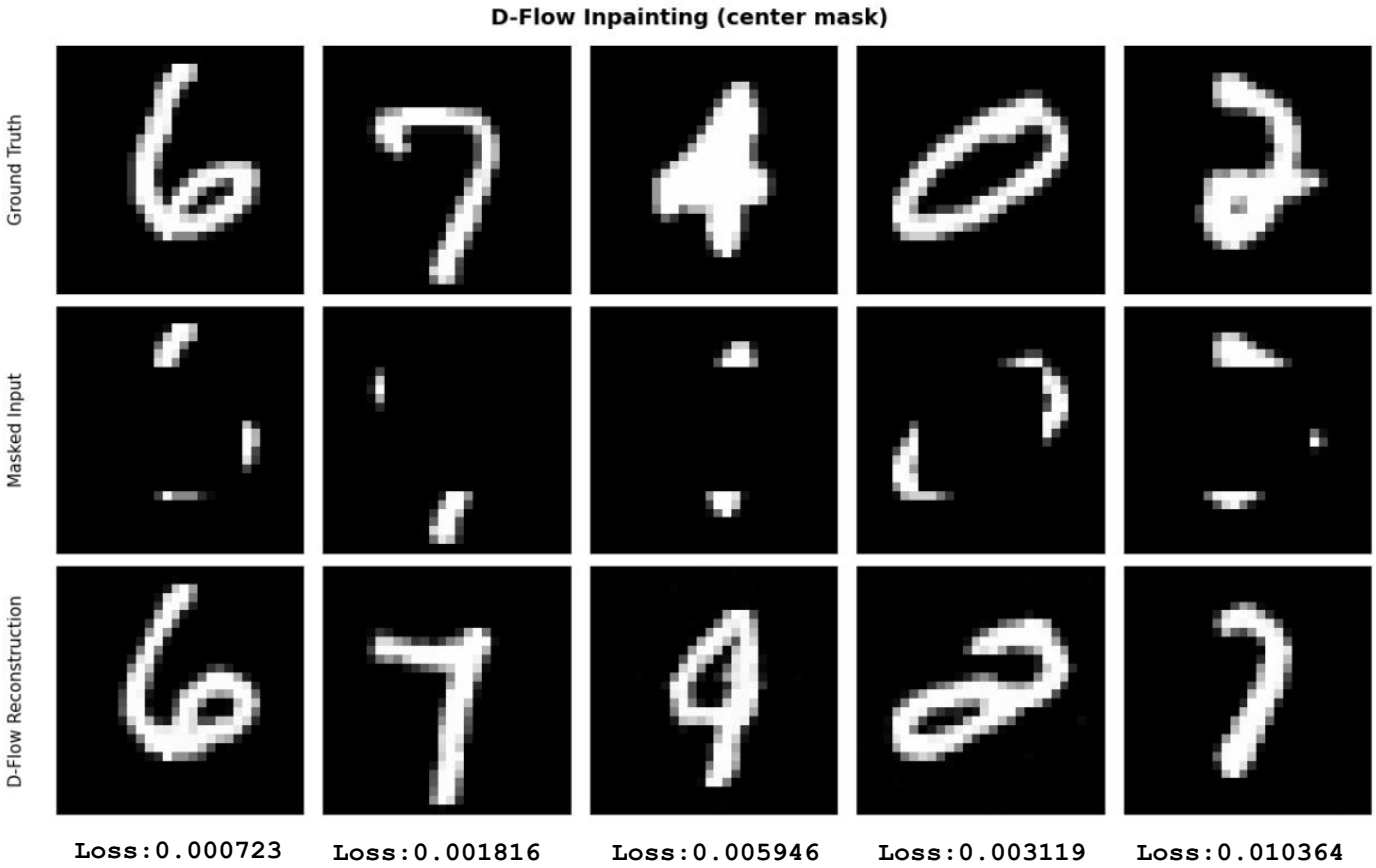
Task	Operator $\mathcal{A}$
Inpainting	$\mathcal{A}(x) = M \odot x$ (binary mask)
Deblurring	$\mathcal{A}(x) = k * x$ (Gaussian convolution)
Super-resolution	$\mathcal{A}(x) = \downarrow_s x$ (downsampling) **(avg pool 2d)**

Evaluate and visualize the effect of the number of D-Flow optimization iterations on the performance of image restoration tasks namely inpainting, deblurring, and super-resolution on the MNIST and STL-10 datasets

INPAINTING

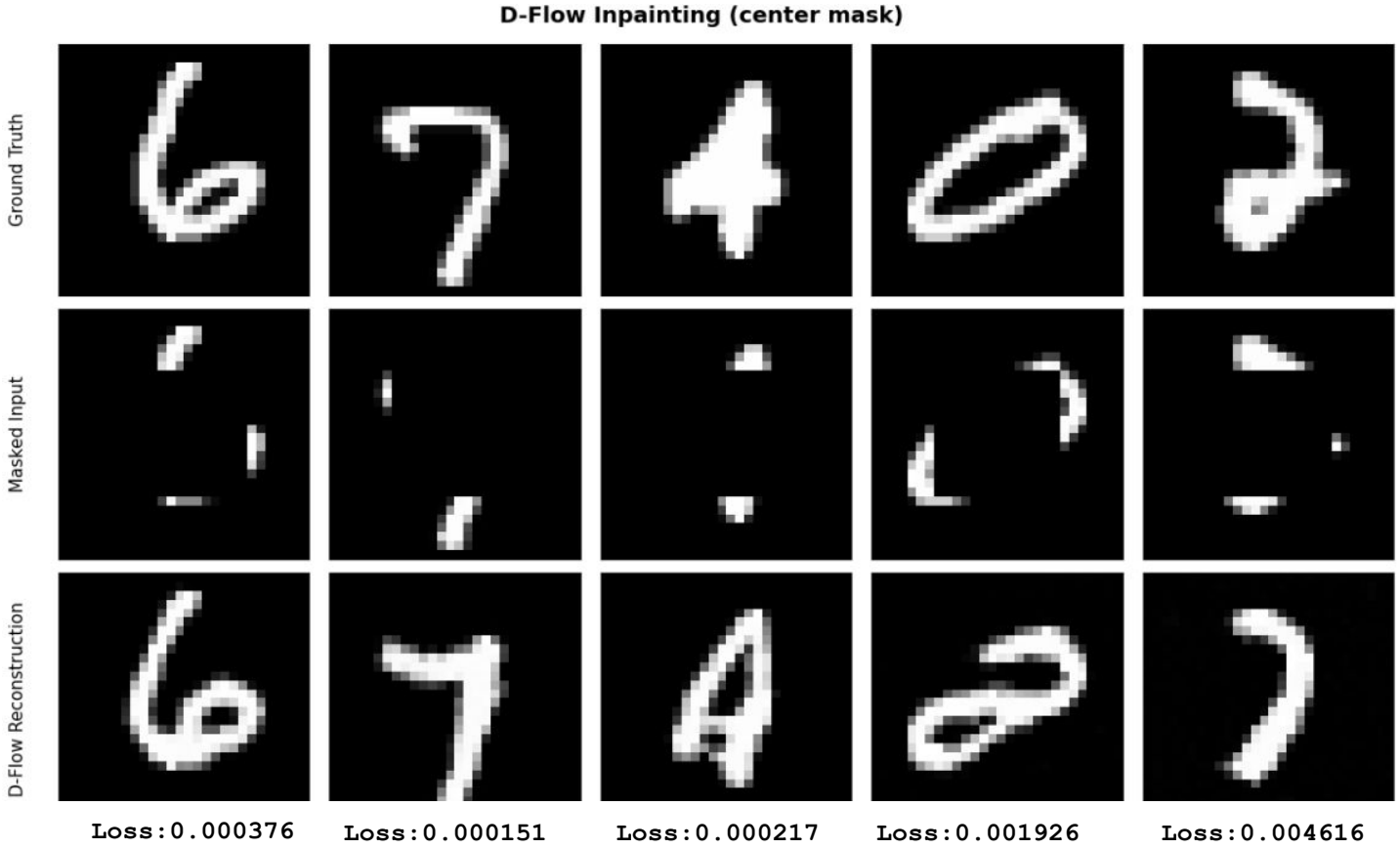
MNIST-100 iterations

Mean PSNR : 12.43 dB



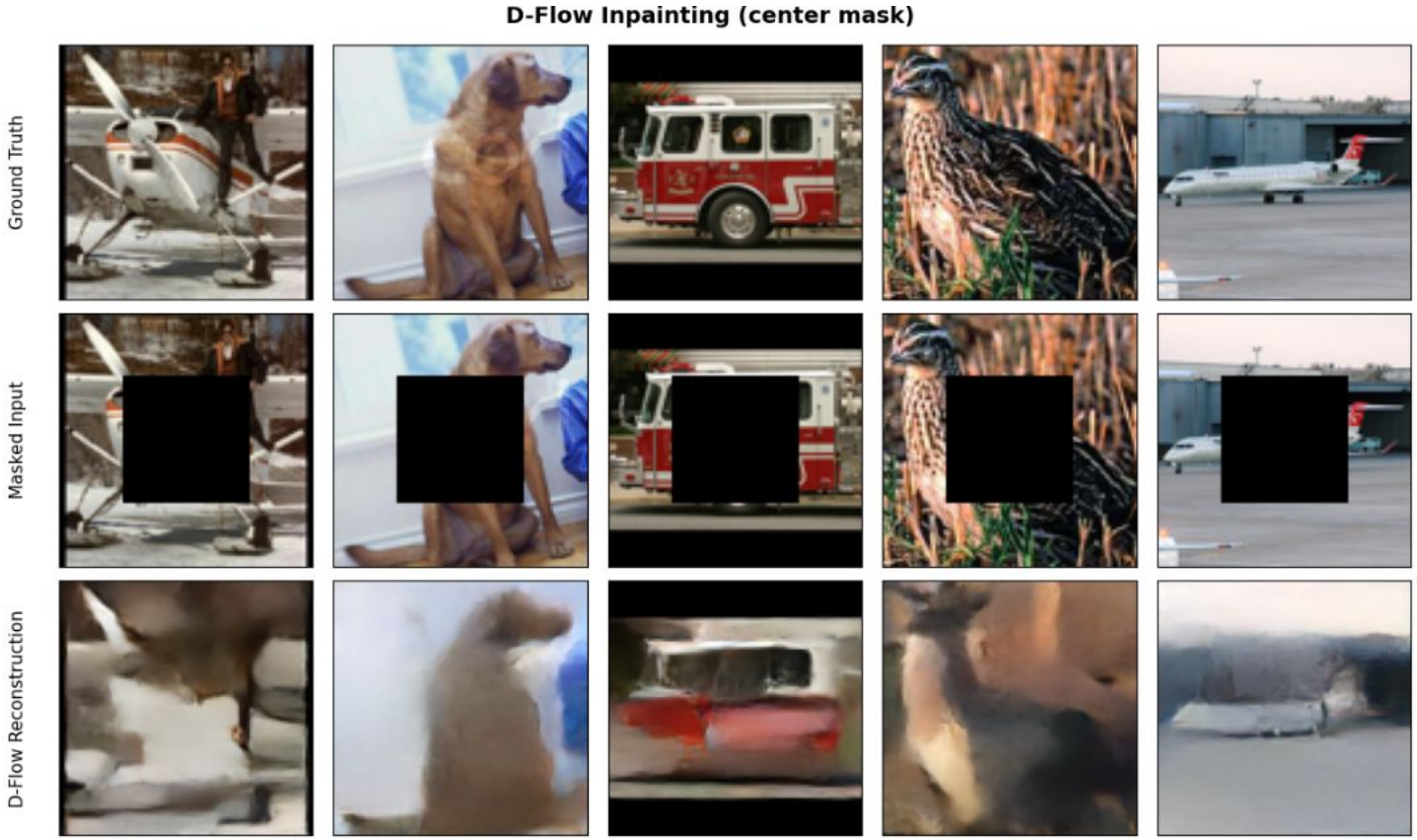
MNIST-200 iterations

Mean PSNR : 13.35 dB



STL10-100 iterations

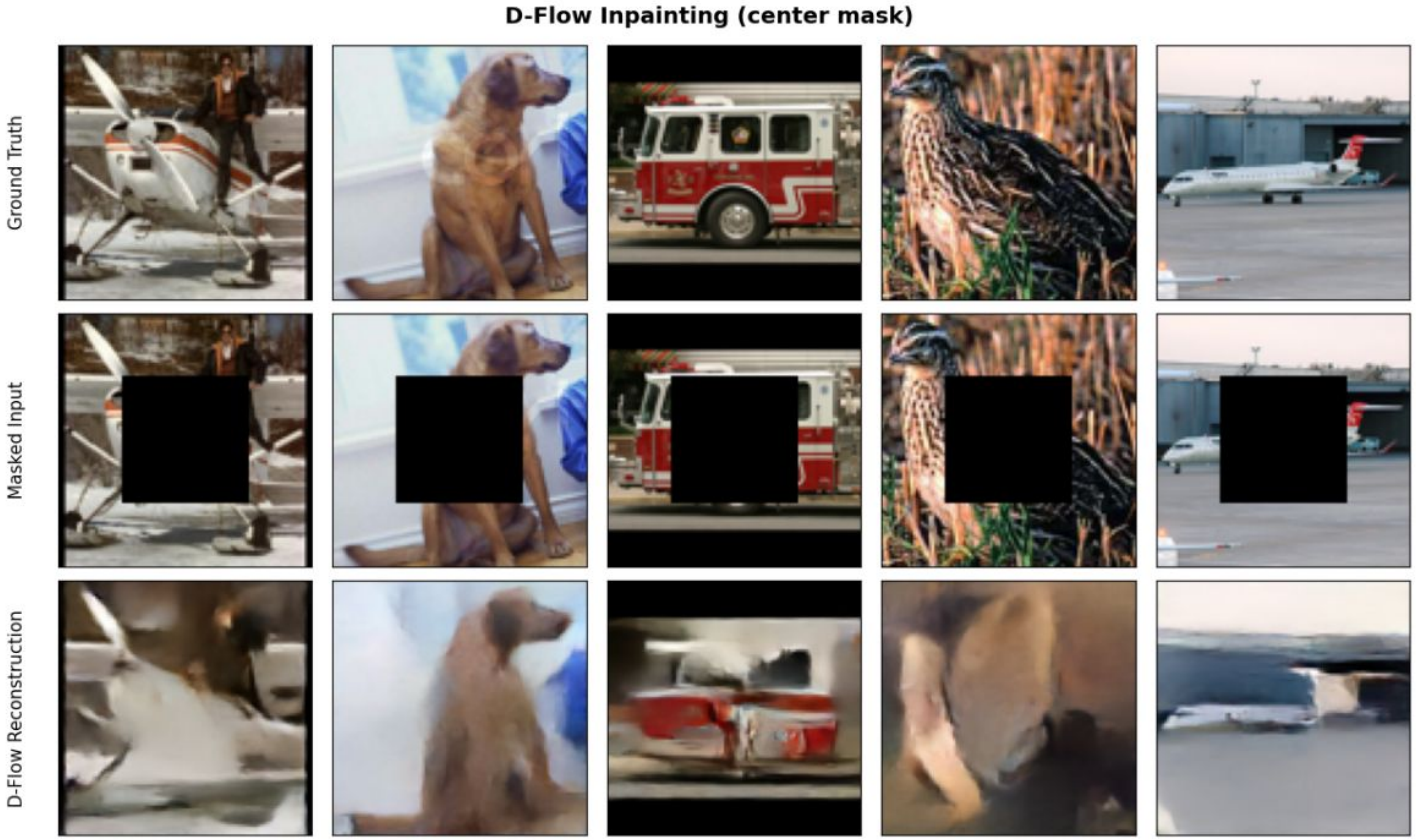
Mean PSNR : 17.38 dB



Loss:0.009103 Loss:0.004637 Loss:0.006765 Loss:0.018070 Loss:0.004849

STL10-200 iterations

Mean PSNR : 17.57 dB

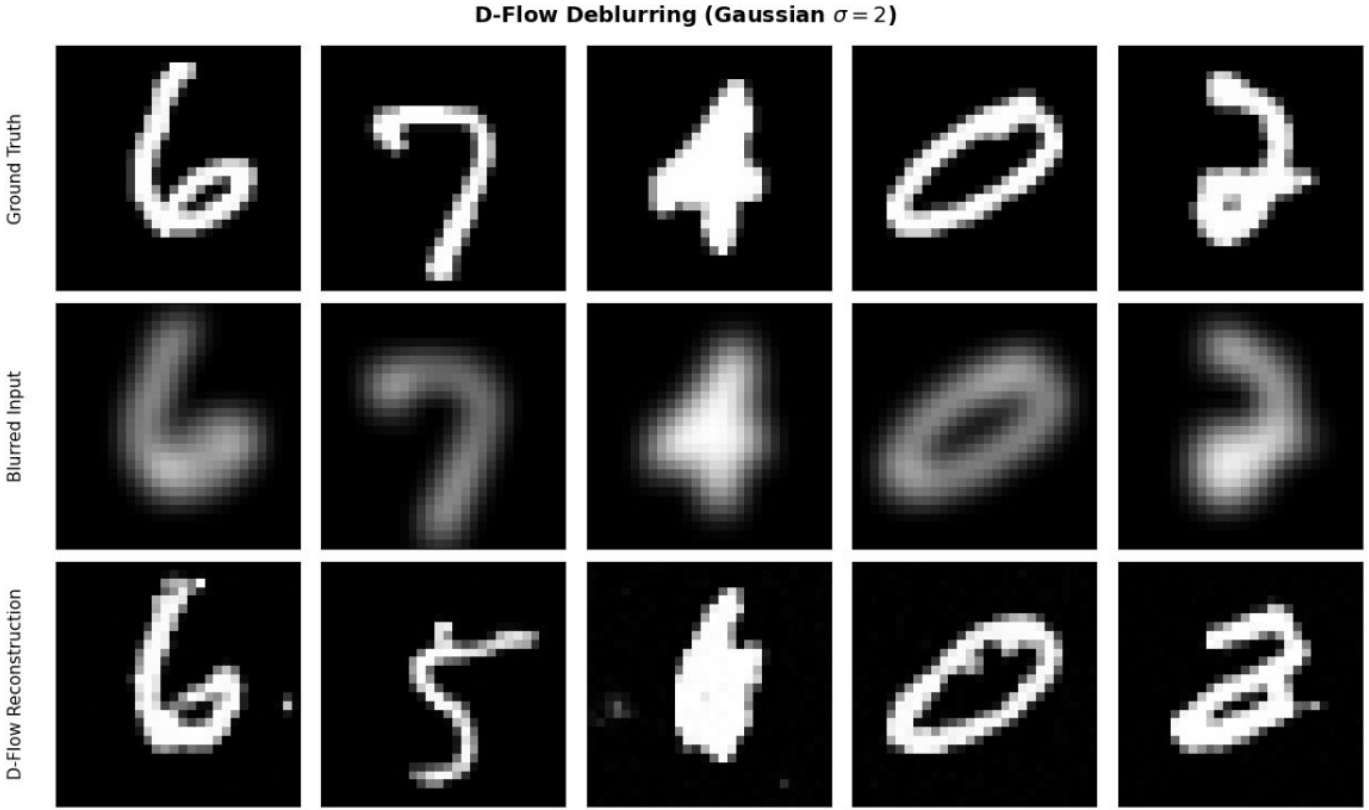


Loss:0.006684 Loss:0.003453 Loss:0.005408 Loss:0.018683 Loss:0.002267

DEBLURRING

MNIST-100 iterations

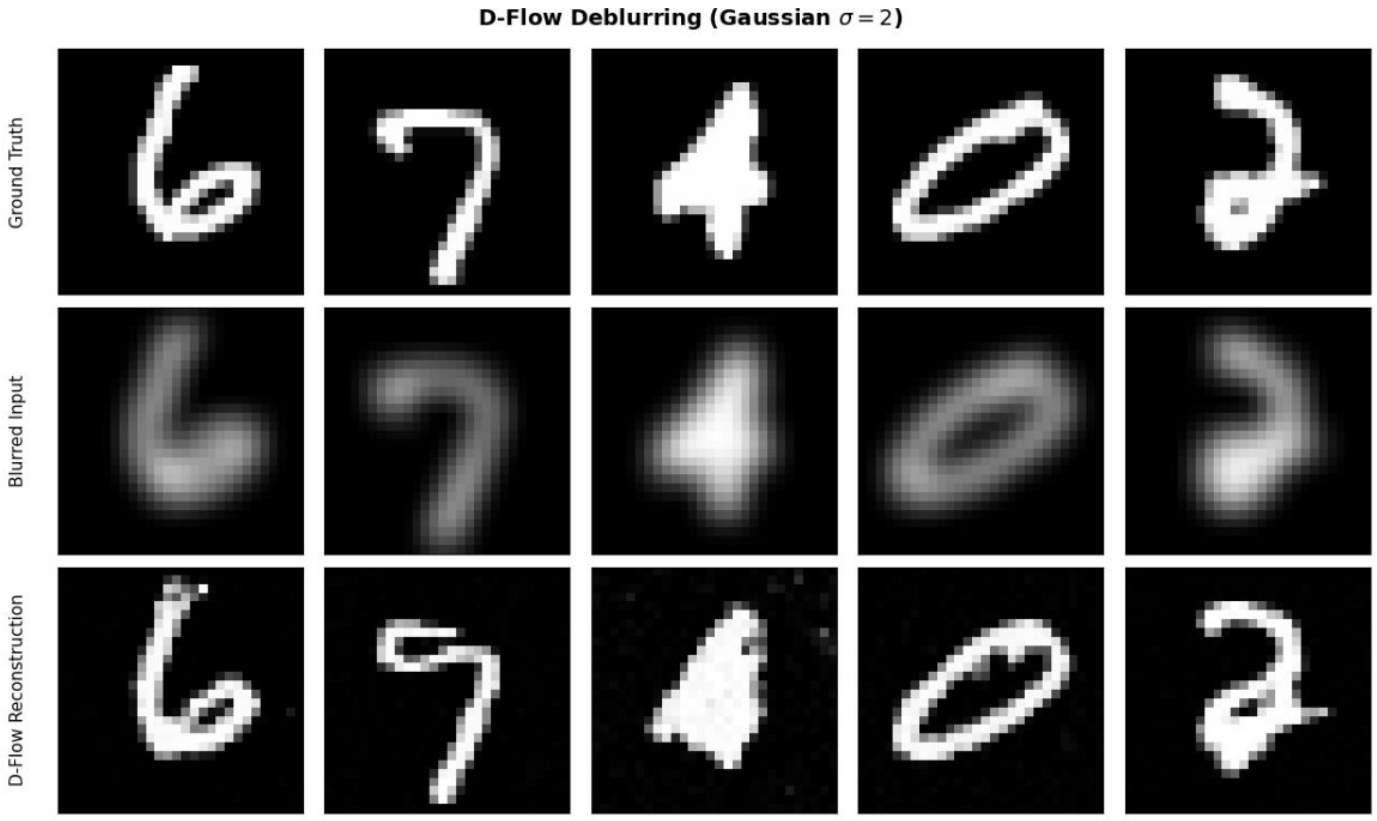
Mean PSNR : 13.49 dB



Loss:0.001157 Loss:0.017007 Loss:0.006618 Loss:0.000817 Loss:0.012298

MNIST-200 iterations

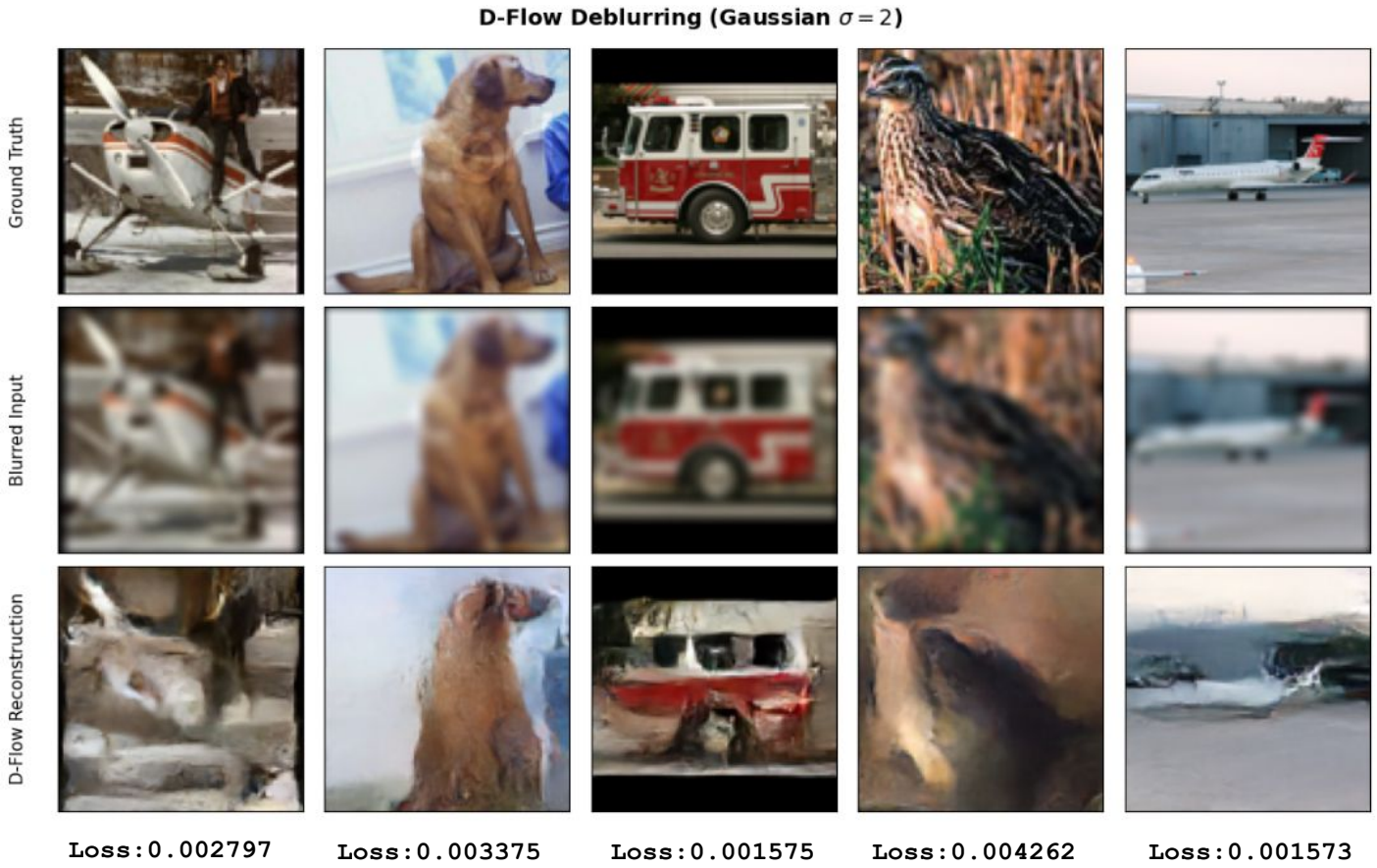
Mean PSNR : 17.59 dB



Loss:0.000142 Loss:0.001429 Loss:0.001680 Loss:0.000135 Loss:0.001684

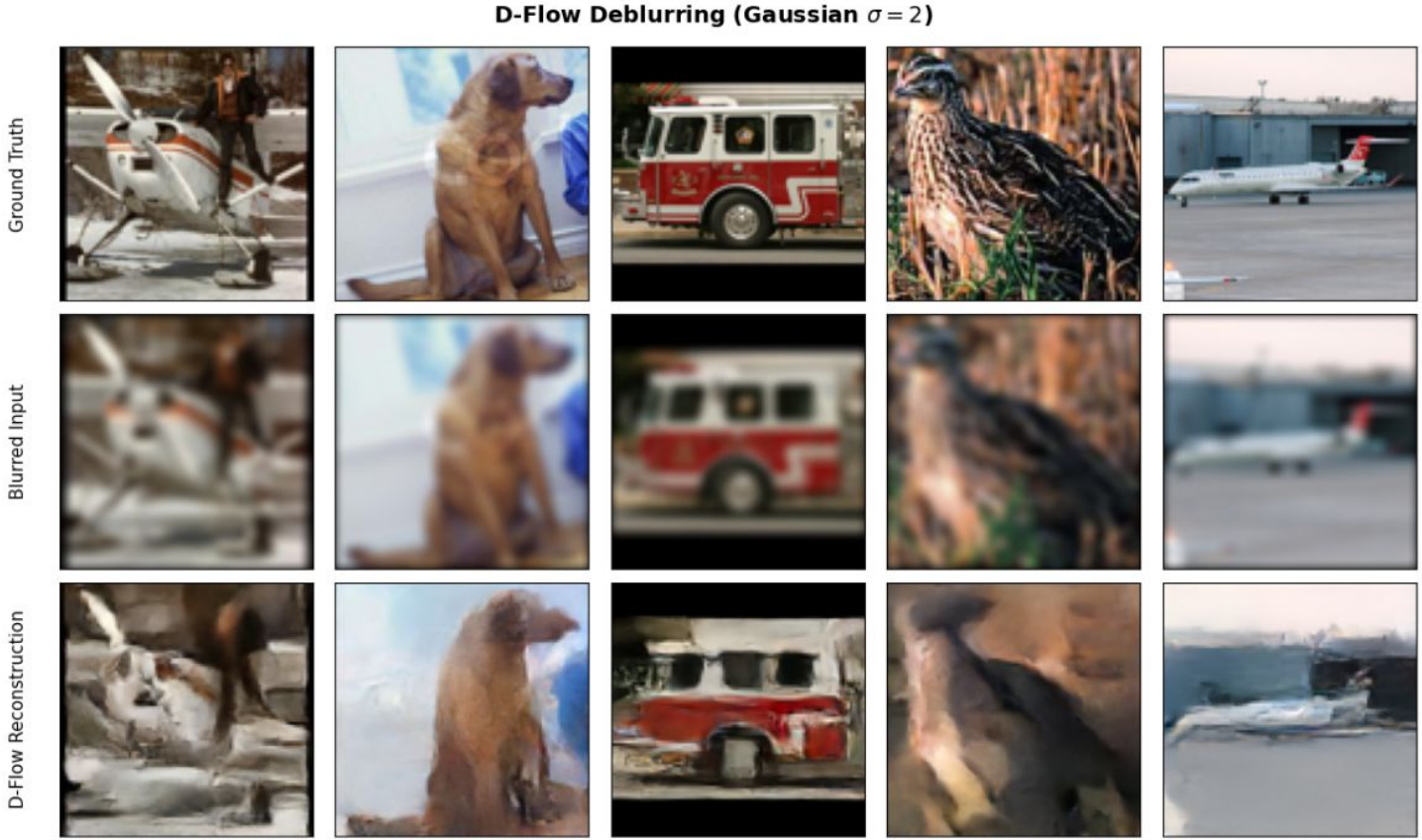
STL10-100 iterations

Mean PSNR : 18.08 dB



STL10-200 iterations

Mean PSNR : 19.36 dB

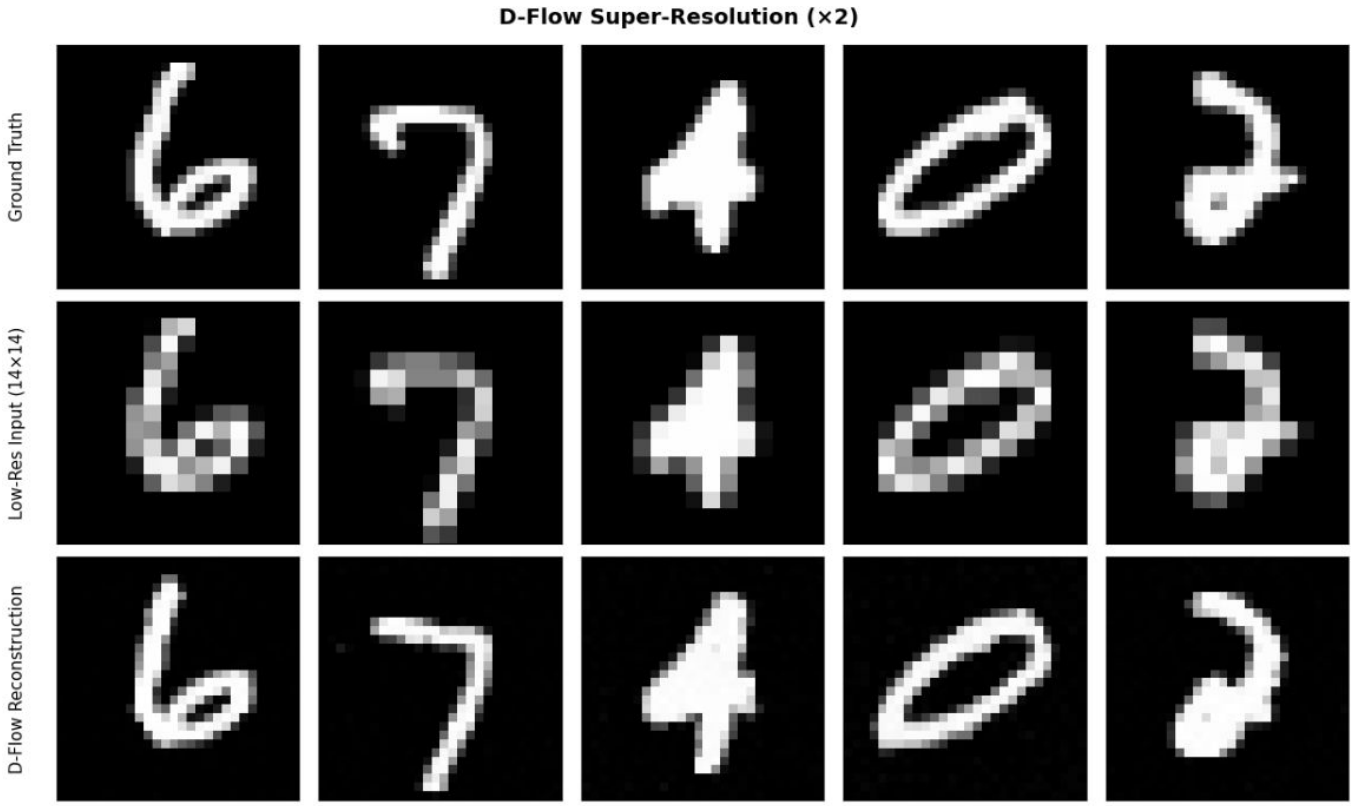


Loss:0.000792 Loss:0.001440 Loss:0.000731 Loss:0.002494 Loss:0.000868

SUPER RESOLUTION

MNIST-100 iterations

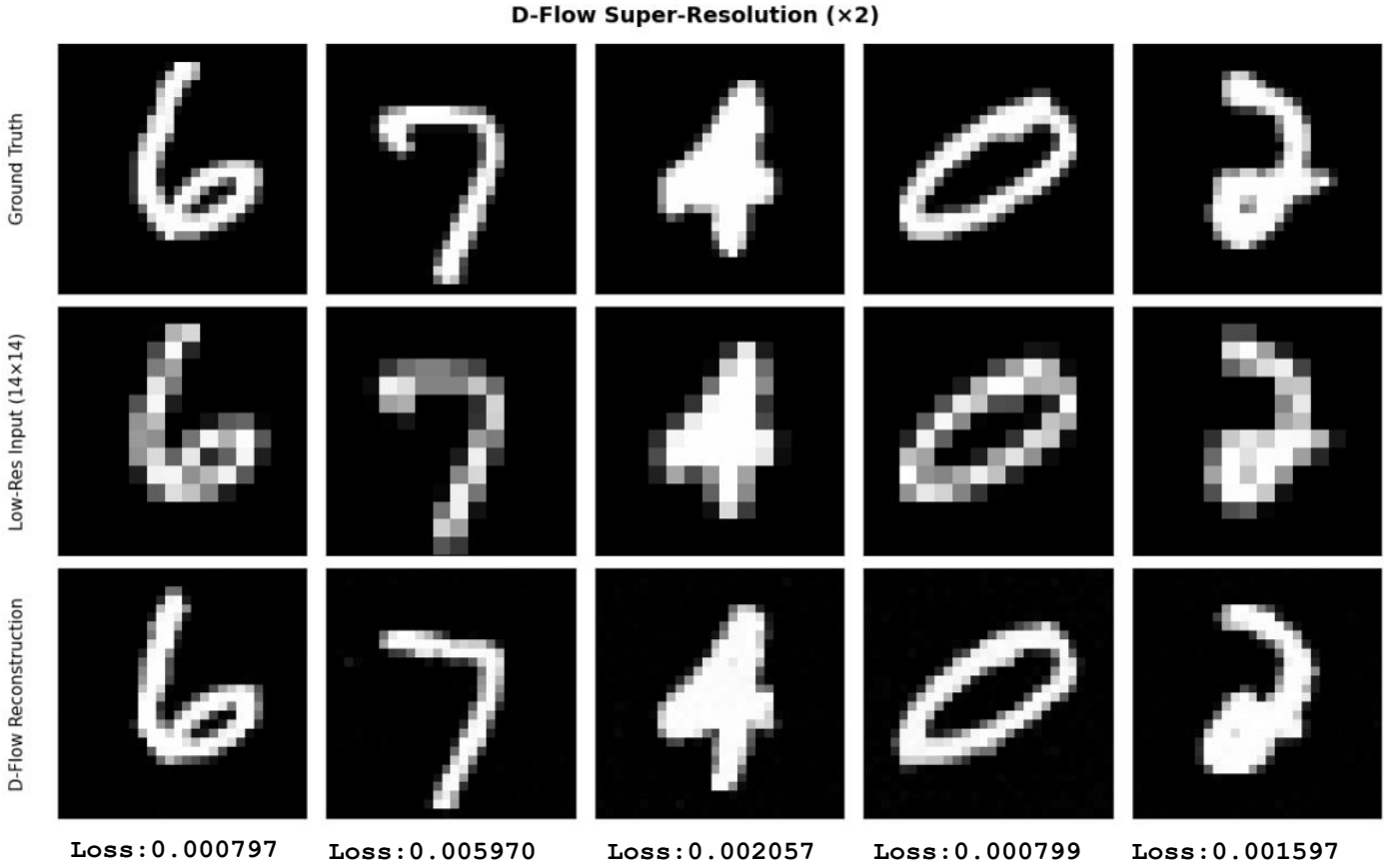
Mean PSNR : 19.38 dB



Loss:0.005464 Loss:0.011702 Loss:0.003078 Loss:0.002793 Loss:0.009606

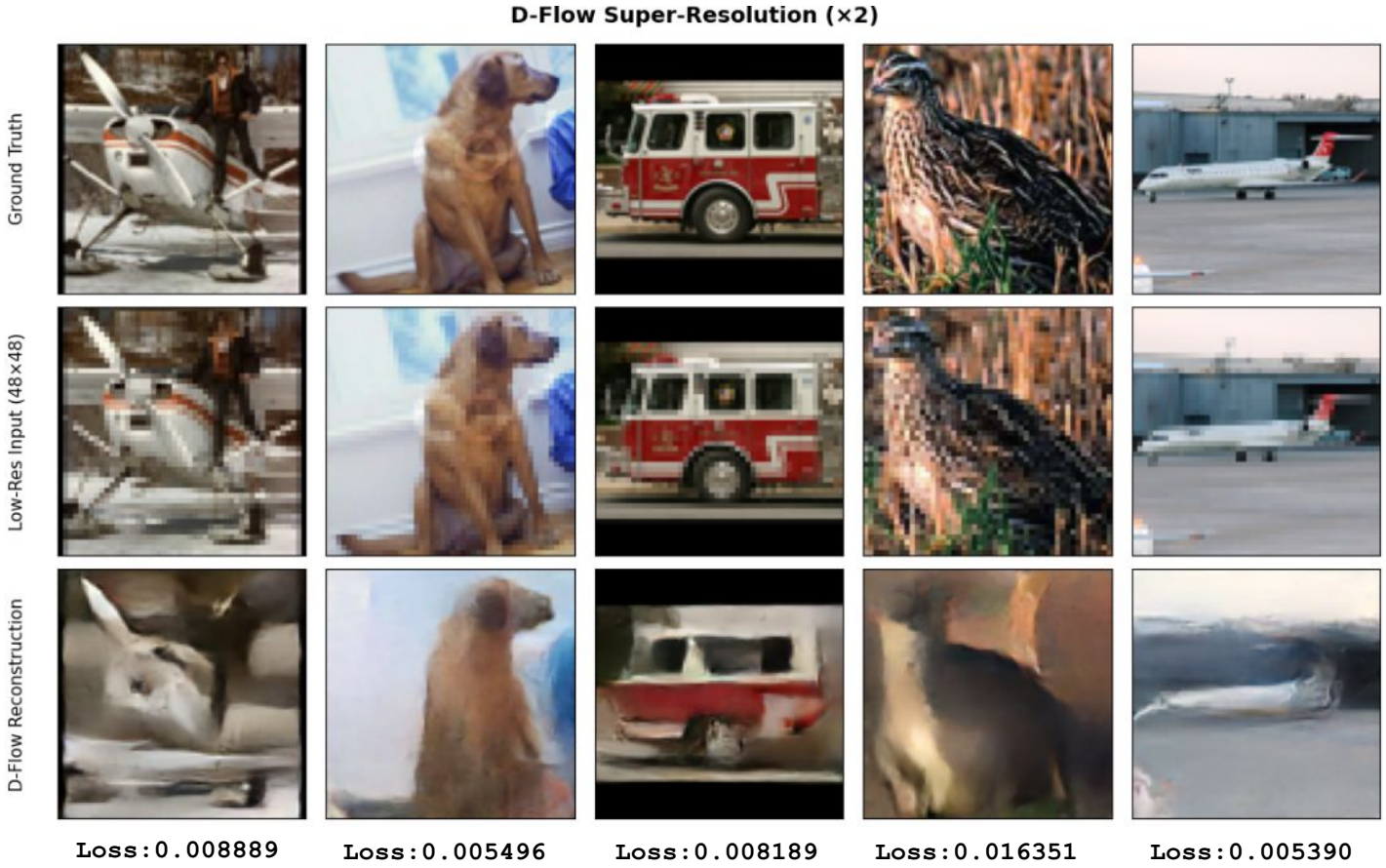
MNIST-200 iterations

Mean PSNR : 23.09 dB



STL10-100 iterations

Mean PSNR : 19.09 dB



STL10-200 iterations

Mean PSNR : 21.19 dB

Thank you for your attention

Questions

Appendix

D-Flow

D-Flow's consists in framing the controlled generation as an **optimization problem** using a differentiable cost function representing the desired control.

Key Observation : For flow models trained with standard Gaussian probability paths, differentiating an arbitrary loss L(x) through the generation process with respect to the initial point x0 projects the gradient onto the major data directions.

→ Implies that defining an arbitrary loss that is convenient to solve the specific problem, and differentiate it w.r.t the initial point x0, allows to achieve a realistic result, changing the task into a classic optimization problem

$$\min_{x_0} \mathcal{L}(x).$$

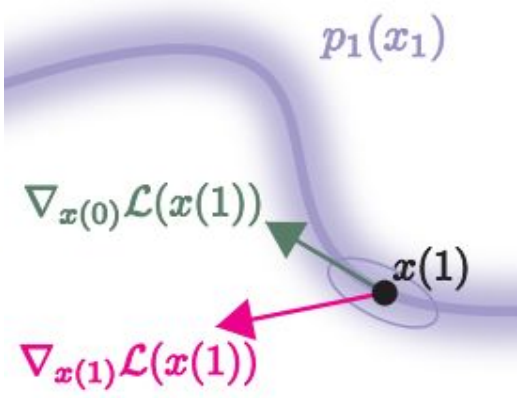
Regularization in D-Flow

3. Implicit Regularization :

The most interesting case is when $R \equiv 0.$

Even without explicit regularization, D-Flow introduces an implicit prior through differentiation of the ODE solution.

Using chain rule, $\nabla_{x_0} L(x(1)) = (D_{x_0} x(1))^\top \nabla_{x(1)} L(x(1)).$



$$\delta x(1) = \frac{d}{d\tau}\bigg|_{\tau=0} \Phi(x_0 - \tau \nabla_{x_0}\mathcal{L}(x(1))) \quad , \quad x(1) = \Phi(x_0)$$

$$= -D\Phi(x_0)\, \nabla_{x_0}\mathcal{L}(x(1))$$

$$= -D\Phi(x_0) \left(D\Phi(x_0)^T \nabla_{x(1)}\mathcal{L}(x(1)) \right)$$

$$\boxed{\delta x(1) = -\left[\underbrace{D_{x_0} x(1)\, D_{x_0} x(1)^T}_{\text{Projection Matrix}} \right] \nabla_{x(1)}\mathcal{L}(x(1))}$$

Practical implementation

<u>Choice of algorithms</u>

- **Initialize x0**

→ sample from the source distribution (normal Gaussian)
or a blend with the backward solution when there is a strong observed signal

- **The solver**

→ *torchdiffeq* package (ODE solver)
use gradient checkpointing to reduce memory consumption at the cost of runtime

- **The optimizer**

→LBFGS algorithm with line search in all experiments
Why ? Because the optimization is not stochastic